Lesson 3 - numpy and pandas

READ ME FIRST (theory). Then open the practice notebook.

Contents

Lesson 3 - numpy and pandas

The theory. Read it, then run `practice.ipynb`. These are the two tools we use for data.

Key words

Word	Simple meaning
numpy	fast math on lists of numbers (arrays)

Word	Simple meaning
array	a row/grid of numbers
pandas	tables of data (like Excel)
DataFrame	a pandas table (we call it `df`)
Series	one column of a table
NaN	an empty / missing cell

---

1. numpy - fast math on many numbers

A numpy **array** is a list of numbers that does math fast and all at once.

```
import numpy as np
a = np.array([10, 20, 30])
a + 5        # [15 25 35]  -> adds 5 to EACH number
```

Why: in ML we do math on thousands of numbers. numpy does it in one line, very fast.

Common mistake: trying to add 5 to a normal Python list `[10,20,30] + 5` -> error. Use a numpy array.

---

2. Boolean mask - keep only what you want

A comparison on an array gives True/False, and we use it to filter.

```
a = np.array([10, 20, 30, 40])
a[a > 25]    # [30 40]  -> keep only numbers bigger than 25
```

Why: we often want only the rows that match a rule.

---

3. pandas - tables of data

A **DataFrame** (`df`) is a table: rows and named columns. We load data with `read_csv`.

```
import pandas as pd
df = pd.read_csv("data.csv")
df.head()      # show first 5 rows
df.shape       # (rows, columns)
df.info()      # column names + types
```

Why: real data lives in tables. pandas is how we open and work with them.

---

4. Pick a column, filter rows

- One column: `df["city"]` (this is a Series).
- Filter rows: keep only the rows that match a rule.

```
df["tenure"]                    # one column
df[df["tenure"] < 6]            # only rows where tenure is under 6
```

Common mistake: `df["tenure"] < 6` alone just gives True/False. Put it inside `df[ ... ]` to actually keep the rows.

---

5. Missing values

An empty cell is **NaN**. Count them per column:

```
df.isna().sum()       # how many missing in each column
```

Why: models crash on missing values. First we must find them (Lesson 4 fixes them).

---

6. groupby - summarise by category

"For each city, what is the average charge?"

```
df.groupby("city")["charges"].mean()
```

Why: this answers business questions fast ("which group is biggest / most expensive?").

---

7. value_counts - count each category

```
df["Contract"].value_counts()     # how many of each contract type
```

---

8. Which chart to use?

You want to…	Use
see the shape of one number column	Histogram
count each category	Bar chart
see outliers (strange far values)	Box plot
see the link between two numbers	Scatter plot

Why: the right chart makes the pattern easy to see. The wrong chart hides it.

Quick summary

- **numpy** = fast math on arrays; use a **mask** (`a[a>25]`) to filter.
- **pandas** = tables (`df`). `read_csv`, `.head()`, `.shape`, `.info()`.
- Pick a column `df["x"]`, filter rows `df[df["x"] < 6]`.
- `df.isna().sum()` finds missing. `groupby` and `value_counts` summarise.
- Pick the chart that fits the question.

Now do this

Open `practice.ipynb`, run every cell on the real Telco data, and do the assignment.